Session 2

Pod에 IAM 권한 부여하기

IRSA와 Pod Identity 비교

홍창섭

소개

- 홍창섭 (https://hongchangsub.com)
- aews 4기
- 가비아 Solution Architect (26.02 ~ ing)
- jam2in Backend Engineer (23.04 ~ 26.01)

발표 주제

- EKS Pod가 AWS 리소스에 접근하는 방법들 (시간 순으로)
- IRSA와 Pod Identity 간의 운영 및 네트워크 관점에서의 차이

Pod은 왜 AWS 자격증명이 필요한가

상황

EKS Pod가 S3, 등 AWS API 호출 필요

- 모든 AWS API 요청은 IAM 자격증명으로 서명되어야 함
- 자격증명 종류: 영구 자격증명(Access Key) vs 임시 자격증명(STS)
- 영구 자격증명은 유출 시 무효화 전까지 그대로 사용 가능
- 임시 자격증명은 만료 시간이 있고 STS가 발급/순환

핵심 질문

Pod에 자격증명을 어떻게 안전하게 전달할 것인가?

요구사항

- 최소 권한 원칙 (Pod 단위 권한 분리)
- 정적 키 배포 회피 (유출/순환 부담)
- 감사 추적 (CloudTrail에서 누가 무엇을 호출했는가)
- 운영 자동화 가능성 (수동 작업 최소화)

1

STEP 1

Access Key를 컨테이너에 직접 주입

동작 방식

IAM User 생성 → Access Key를 환경변수 / Secret으로 Pod에 주입

```
env:
  - name: AWS_ACCESS_KEY_ID
    valueFrom:
      secretKeyRef:
        name: aws-creds
        key:  access_key_id
```

자격증명은 영구적이며, 명시적으로 비활성화할 때까지 유효

문제점

영구 자격증명

유출 시 회수까지 무제한 사용 가능

순환 비용

키 교체 시 모든 워크로드 재배포 필요

권한 과잉

보통 Pod별이 아닌 공용 키를 사용 → 최소 권한 위반

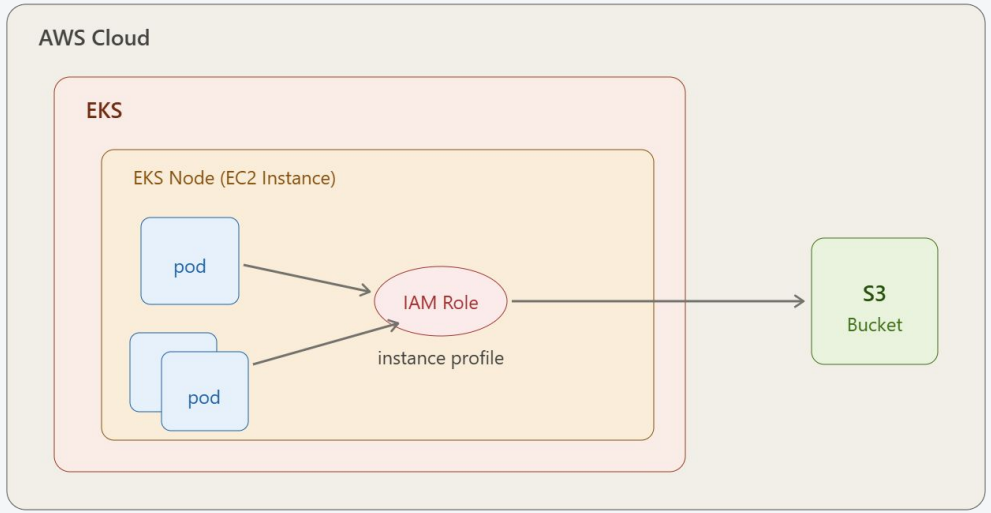
감사 어려움

여러 Pod이 동일 IAM User로 호출 시 주체 식별 곤란

STEP 2

2 EC2 Instance Profile 활용

정적 키 대신 EC2 Instance Profile 사용



- 코드 변경 없이 pod에 권한 부여 가능

노드 단위 권한

같은 노드의 모든 Pod가 동일한 Role 권한 공유

STEP 3

3 IRSA — IAM Roles for Service Accounts (2019)

핵심 아이디어

EKS가 발급한 ServiceAccount JWT를 OIDC 토큰으로 사용해 파드가 IAM에서 임시 자격증명 획득

사전 셋업 단계

- IAM OIDC Identity Provider 등록
- IAM Role 생성 + Trust Policy
- IAM Policy attach
- ServiceAccount + 어노테이션

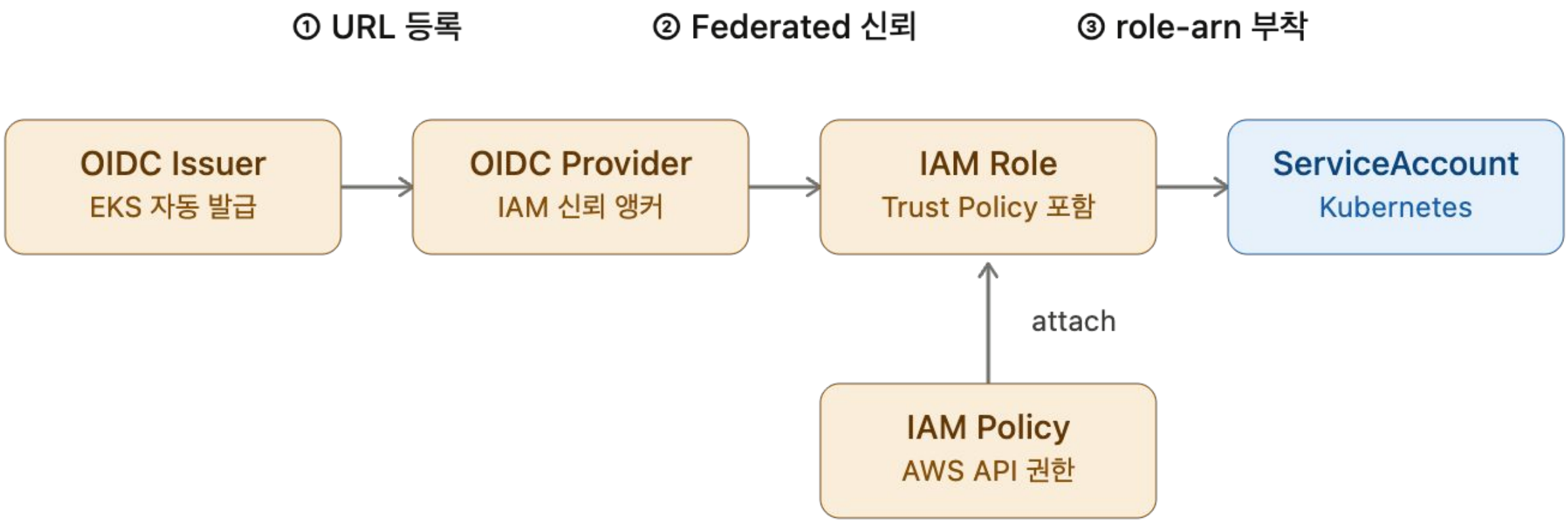
Pod Admission(Mutation)

- Pod 생성 요청
- Pod Identity Webhook이 SA 어노테이션 감지
- Pod에 환경변수 + Token 마운트

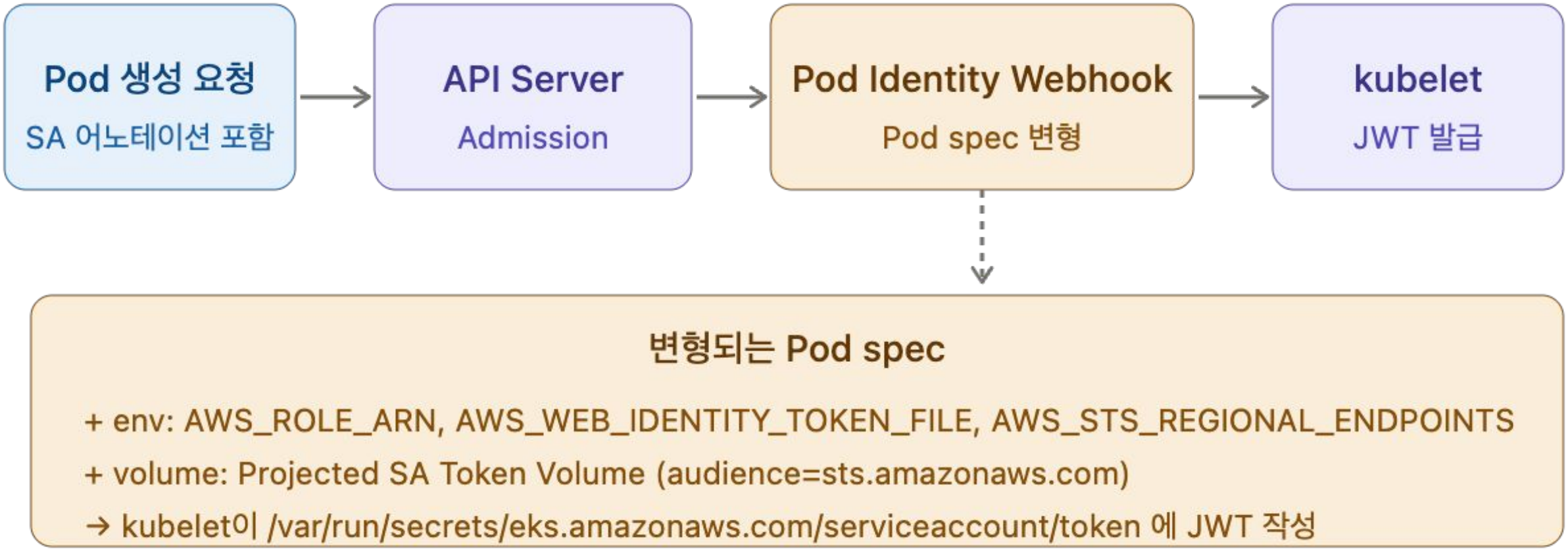
토큰 교환 및 권한 획득

- SDK가 SA 내의 환경변수 및 토큰 감지
- STS AssumeRoleWithWebIdentity 호출
- STS 검증
- 임시 자격 증명 발급 후 리소스 접근

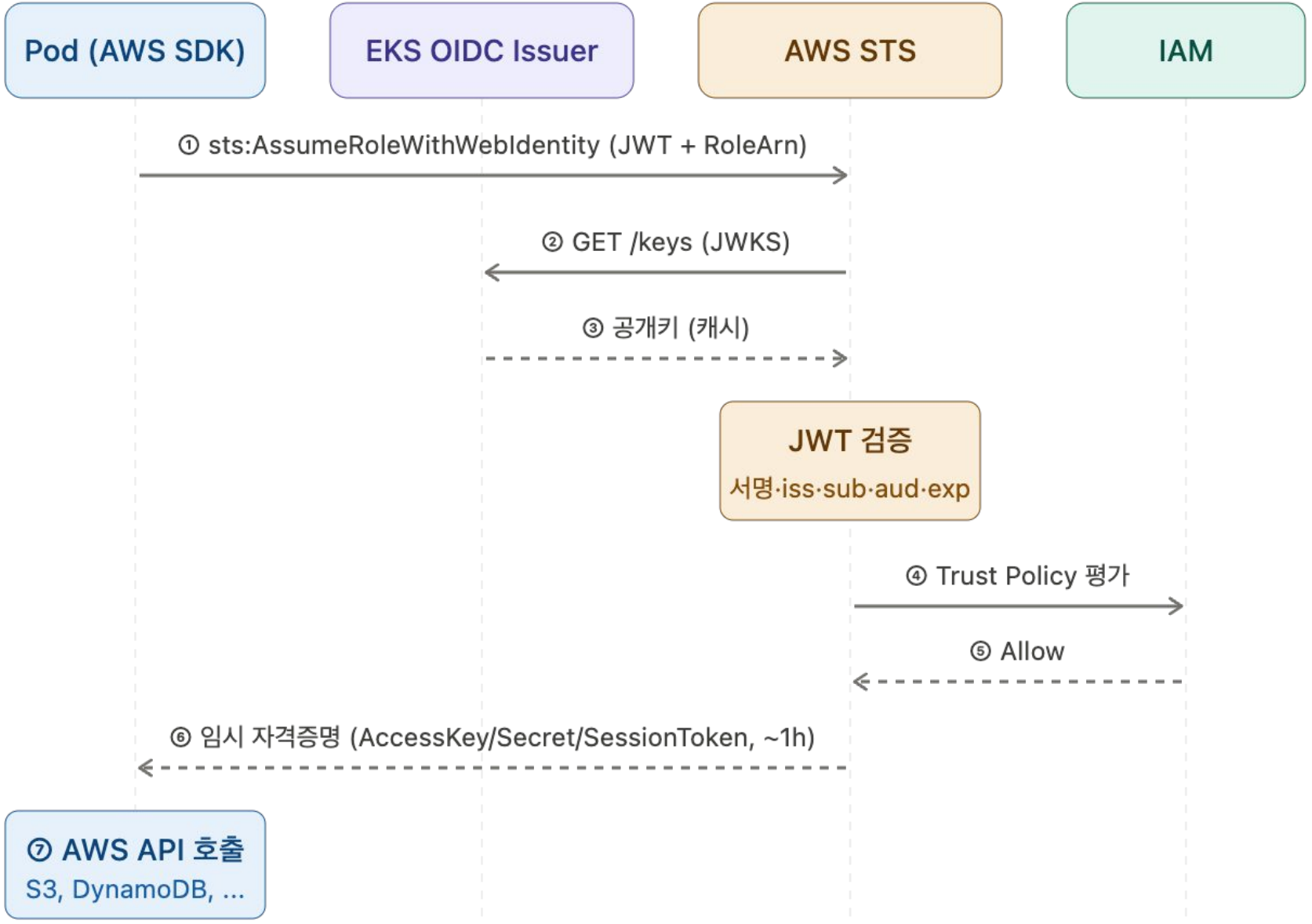
IRSA 동작 흐름 1. 사전 셋업 단계



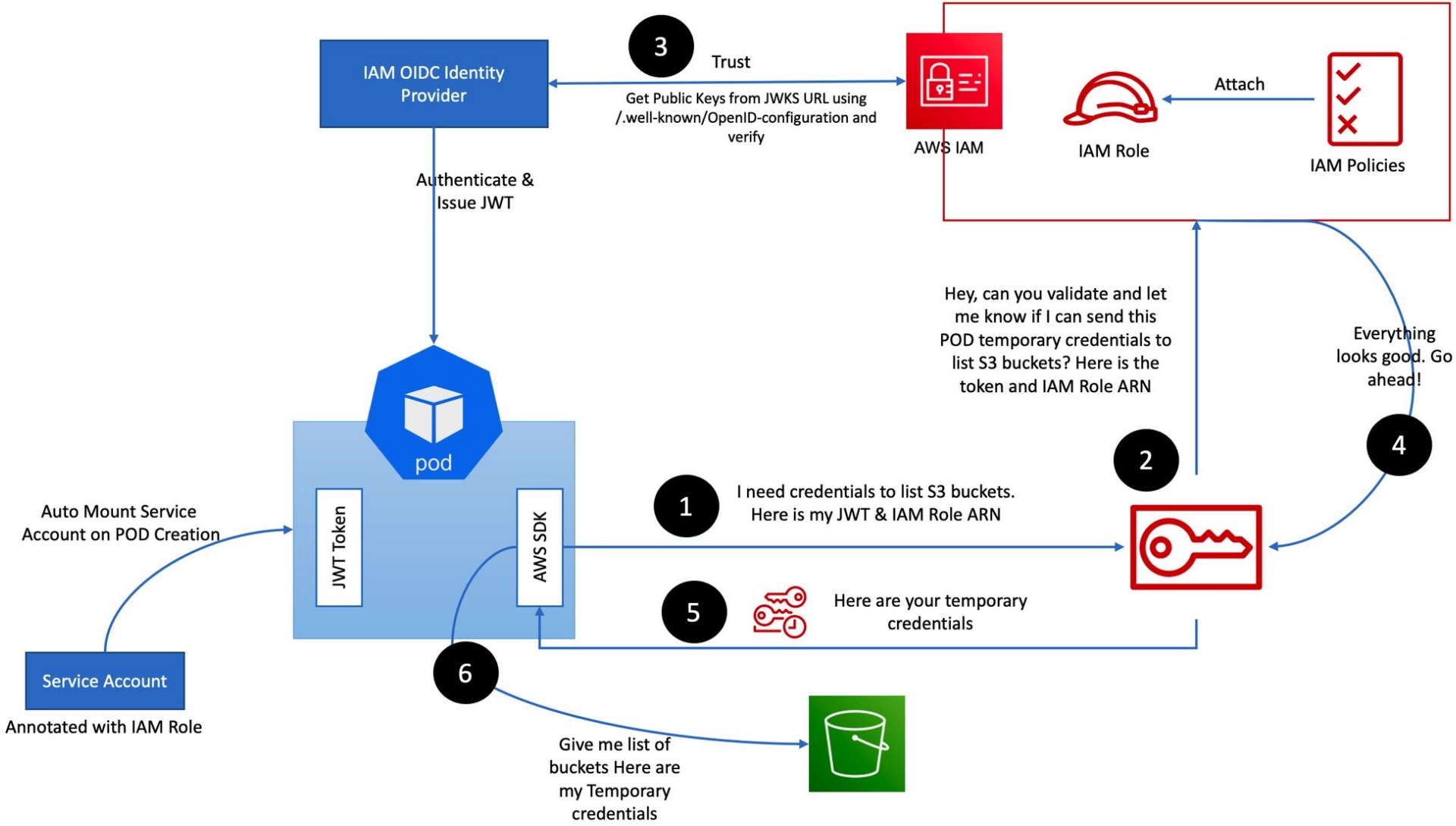
IRSA 동작 흐름 2. Pod Admission (Mutation)



IRSA 동작 흐름 3. 토큰 교환 및 권한 획득

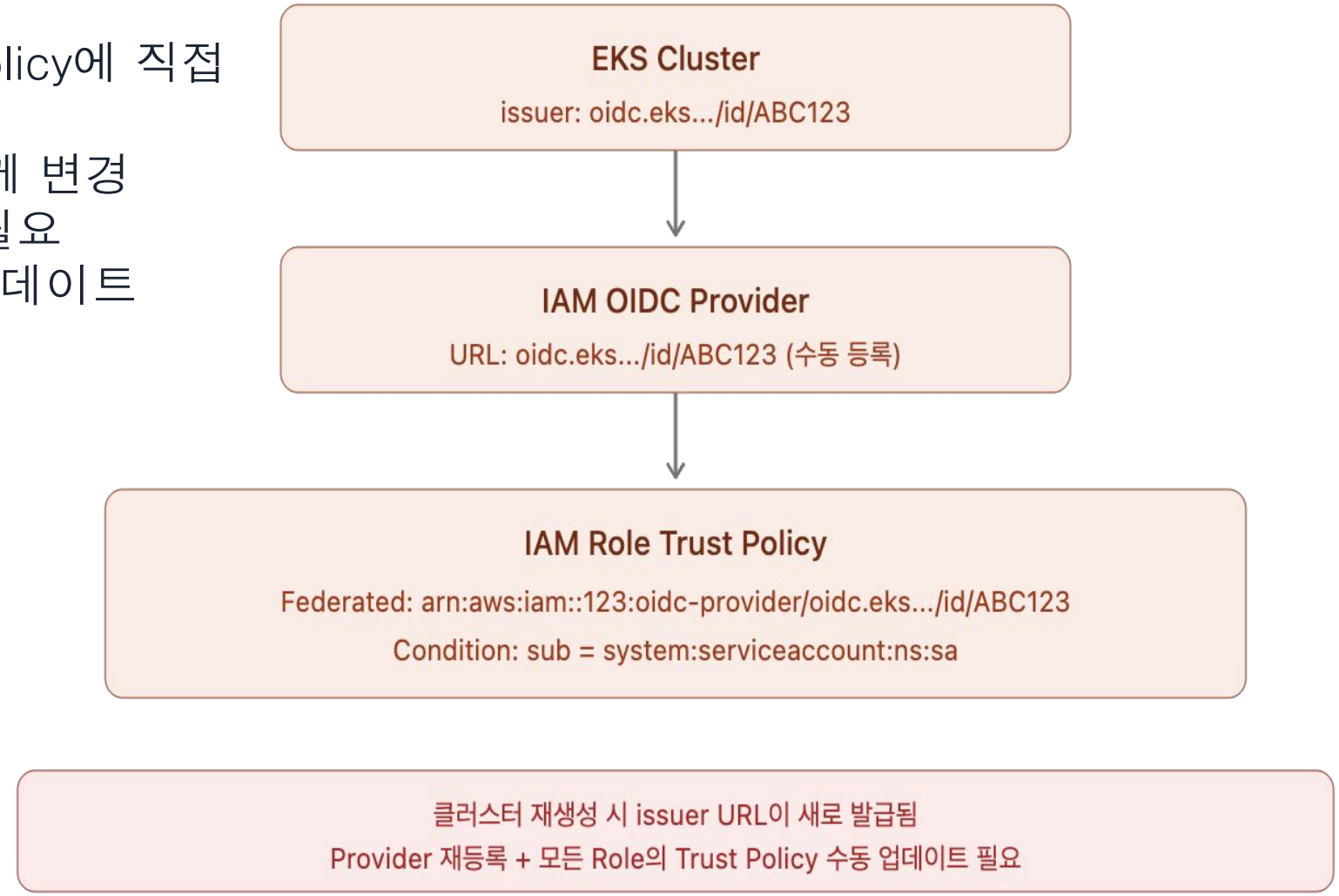


IRSA 전체 동작 흐름

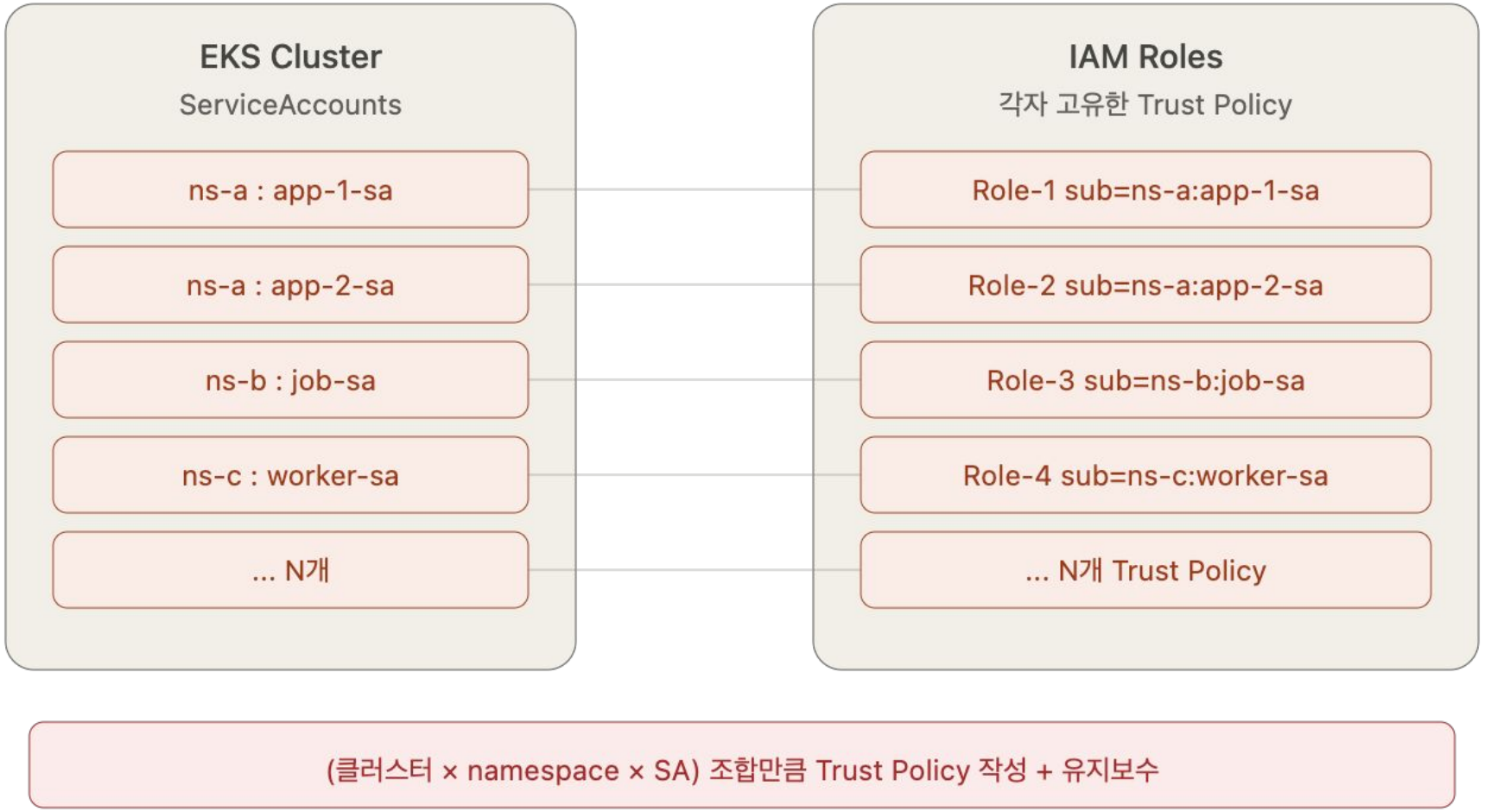

IAM OIDC Identity Provider
3
Trust
Get Public Keys from JWKS URL using /.well-known/OpenID-configuration and verify
AWS IAM
IAM Role
Attach
IAM Policies
Authenticate & Issue JWT
Hey, can you validate and let me know if I can send this POD temporary credentials to list S3 buckets? Here is the token and IAM Role ARN
Everything looks good. Go ahead!
pod
JWT Token
AWS SDK
Auto Mount Service Account on POD Creation
1
I need credentials to list S3 buckets. Here is my JWT & IAM Role ARN
2
4
5
Here are your temporary credentials
Service Account
Annotated with IAM Role
6
Give me list of buckets Here are my Temporary credentials

IRSA의 운영 한계 1. IdP 관리

- OIDC Issuer URL을 IdP, Trust Policy에 직접 등록
- 해당 URL은 클러스터 변경 시 함께 변경 ->IdP, Trust Policy도 업데이트 필요
- Blue-Green 업데이트 시, 별도 업데이트



IRSA의 운영 한계 2. Trust Policy의 증가



- SA마다 같은 Role을 가져도 NS 다르다면 각 Trust Policy 마다 별도의 SA 매핑 정보 추가
- **(클러스터 * NS * SA)** 조합 만큼 Trust Policy 작성 필요

IRSA Trust Policy 누락

신뢰할 수 있는 엔터티

지정된 조건에서 이 역할을 수임할 수 있는 엔터티입니다.

```
{
    "Version": "2012-10-17",
    "Statement": [
        {
            "Effect": "Allow",
            "Principal": {
                "Federated": "arn:aws:iam::911283464785:oidc-provider/oidc.eks.ap-northeast-2.amazonaws.com/id/032357E88E266F4AE7C2E8CF6F5EFEB0"
            },
            "Action": "sts:AssumeRoleWithWebIdentity",
            "Condition": {
                "StringEquals": {
                    "oidc.eks.ap-northeast-2.amazonaws.com/id/032357E88E266F4AE7C2E8CF6F5EFEB0:sub": "system:serviceaccount:default:my-sa",
                    "oidc.eks.ap-northeast-2.amazonaws.com/id/032357E88E266F4AE7C2E8CF6F5EFEB0:aud": "sts.amazonaws.com"
```

Important

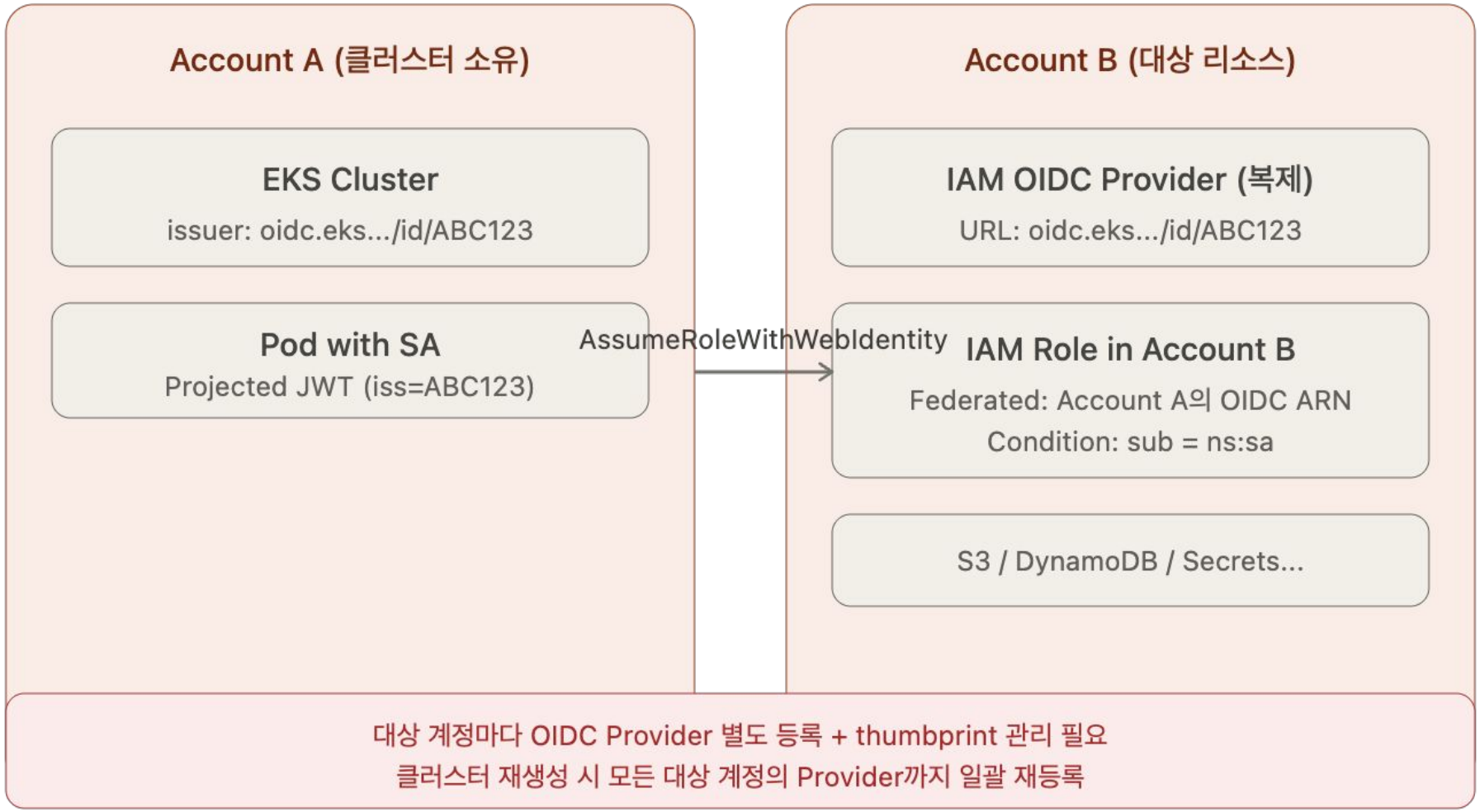
For least privilege, add a `StringEquals` condition for the `sub` claim to restrict this role to a specific Kubernetes service account. Without a `sub` condition, any service account in the cluster can assume this role. The `sub` value uses the format `system:serviceaccount:NAMESPACE:SERVICE_ACCOUNT_NAME`. For example, to restrict to a service account named `my-service-account` in the `default` namespace:

```
"oidc.eks.region-code.amazonaws.com/id/EXAMPLED539D4633E53DE1
```

Trust Policy에 Sub 값이 누락될 경우 Role 권한 탈취 문제

발급된 모든 주체(SA)는 해당 irsa를 assume 해서 Role 획득 가능

IRSA의 운영 한계 3. Cross-Account 시나리오의 복잡성



- Account A 클러스터가 Account B 리소스 접근 시 B 계정에 IdP 등록 필요
- **(클러스터 수 * 대상 계정 수)** 만큼 등록 필요

4

STEP 4

EKS Pod Identity

핵심 변화

신뢰 관계를 IAM Trust Policy(OIDC) → EKS API의 Pod Identity Association으로 이전

신원 매핑

- Pod Identity Agent Add-on 설치 (데몬셋)
- IAM Role 생성
- Pod Identity Association 등록

토큰 주입 (Mutation)

- Pod 구동 전에 Pod에 마운트
- Projected SA Token 주입
- 환경 변수 주입

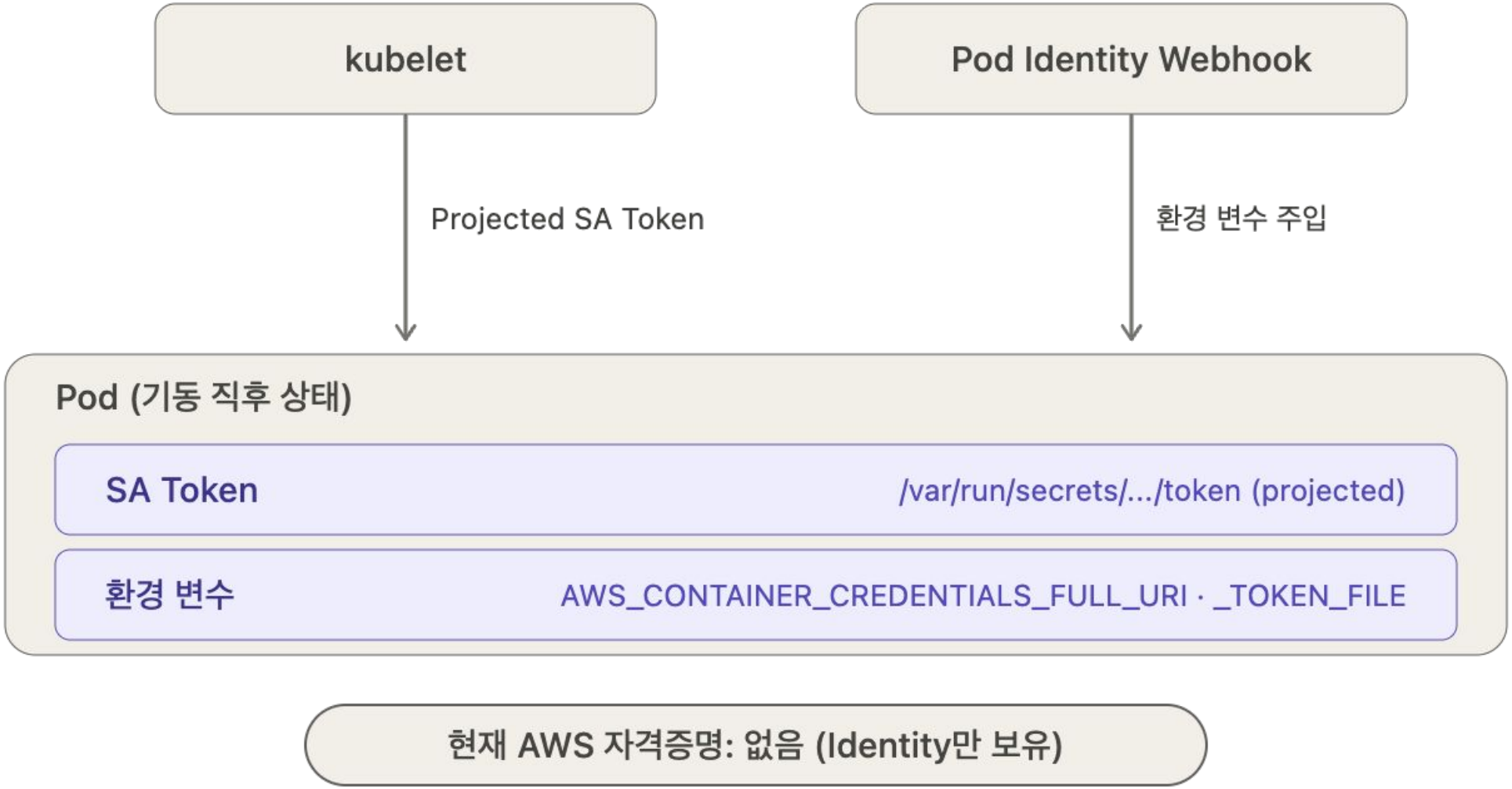
자격증명 교환

- SDK에서 Pod Identity Agent로 요청
- Agent가 EKS Auth API에 assume 호출
- EKS Auth가 토큰 검증
- 임시 자격 증명 발급

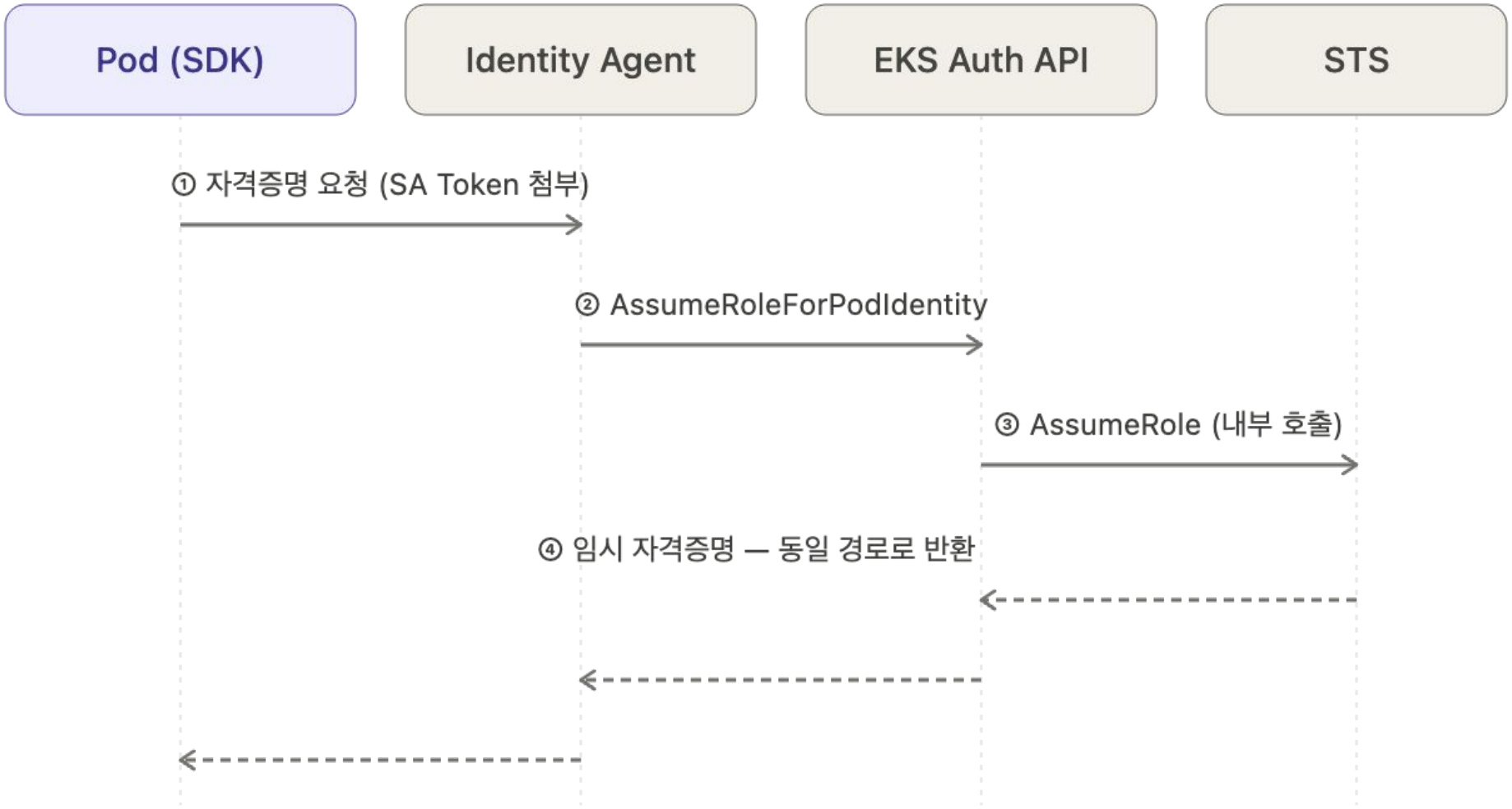
Pod Identity 동작 흐름 1. 신원 매핑



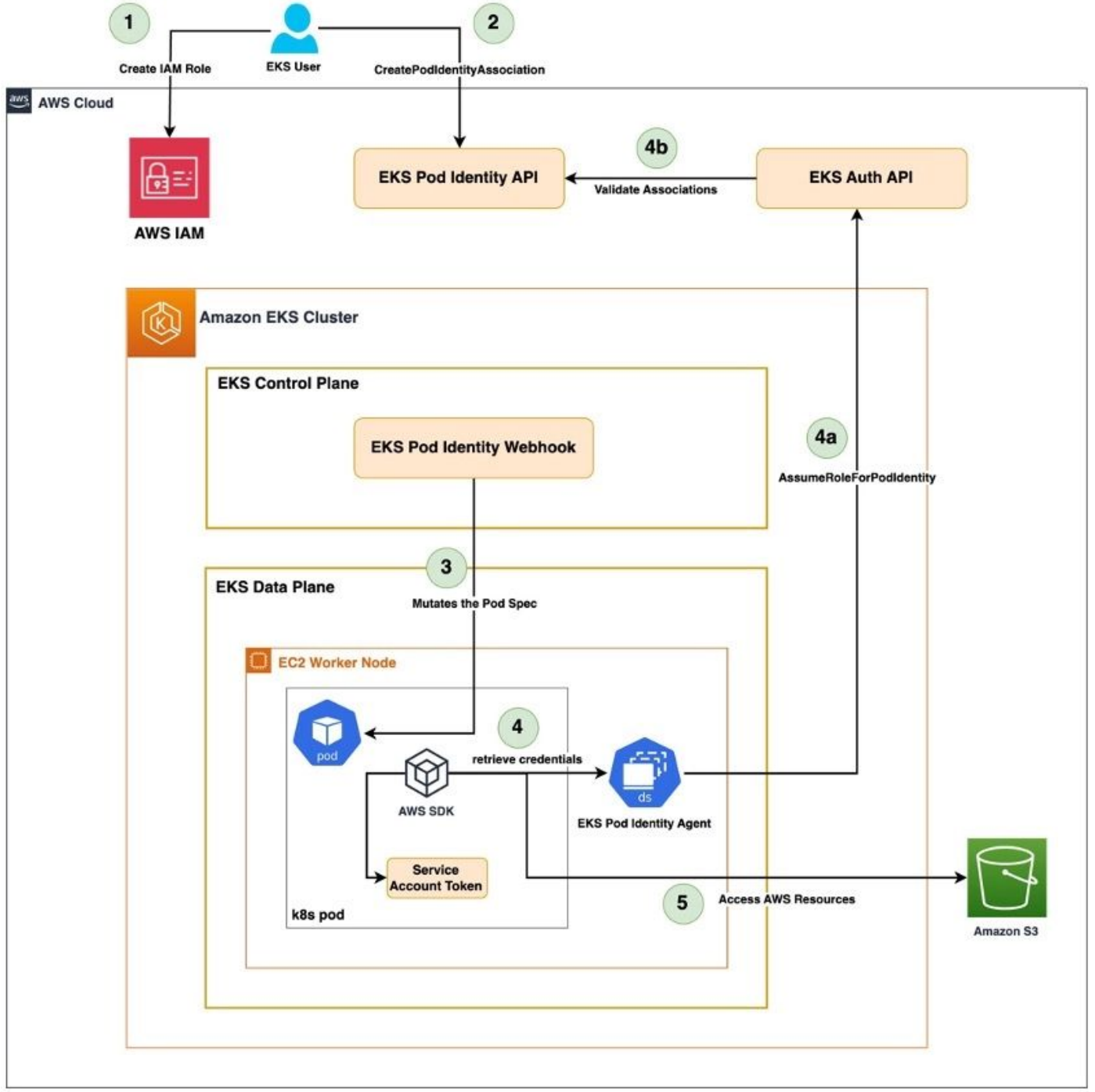
Pod Identity 동작 흐름 2. 토큰 주입



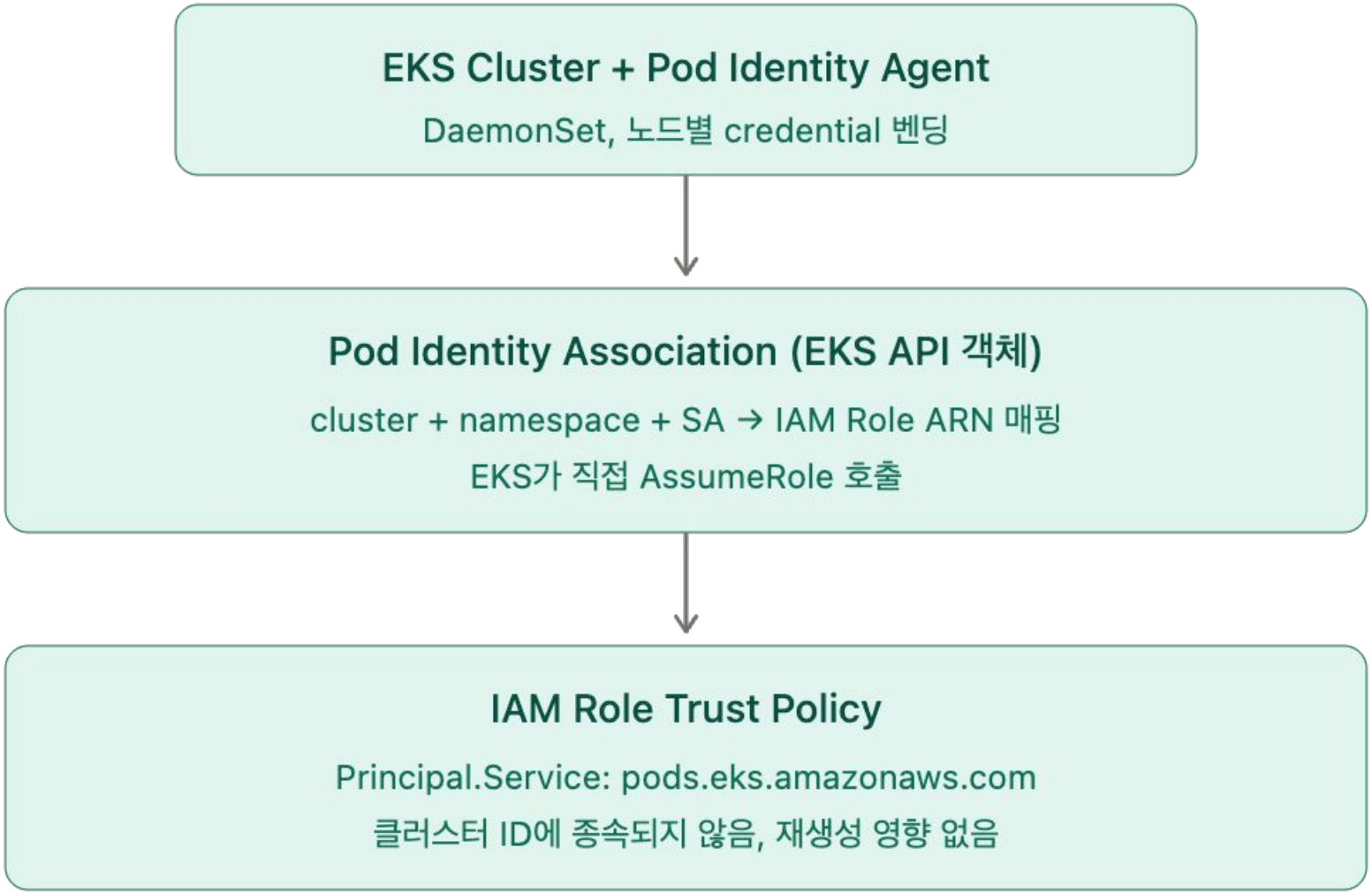
Pod Identity 동작 흐름 3. 자격증명 교환



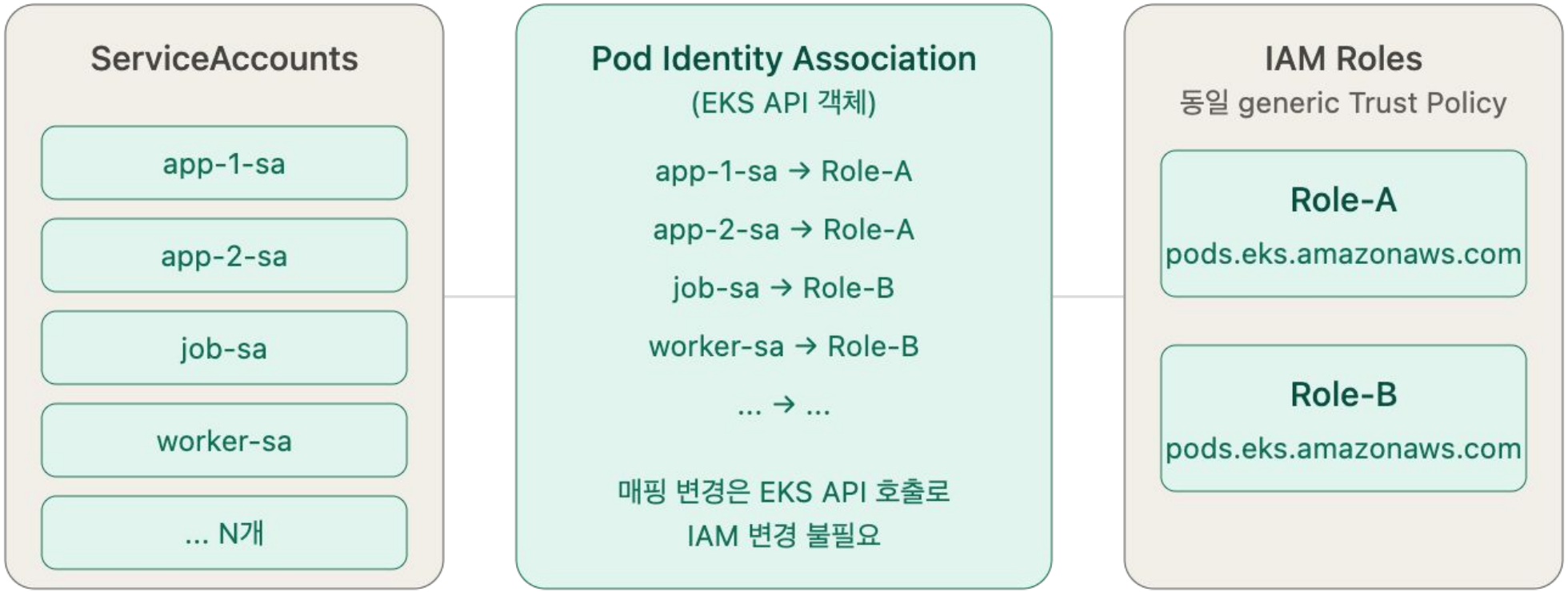
Pod Identity 전체 동작 흐름



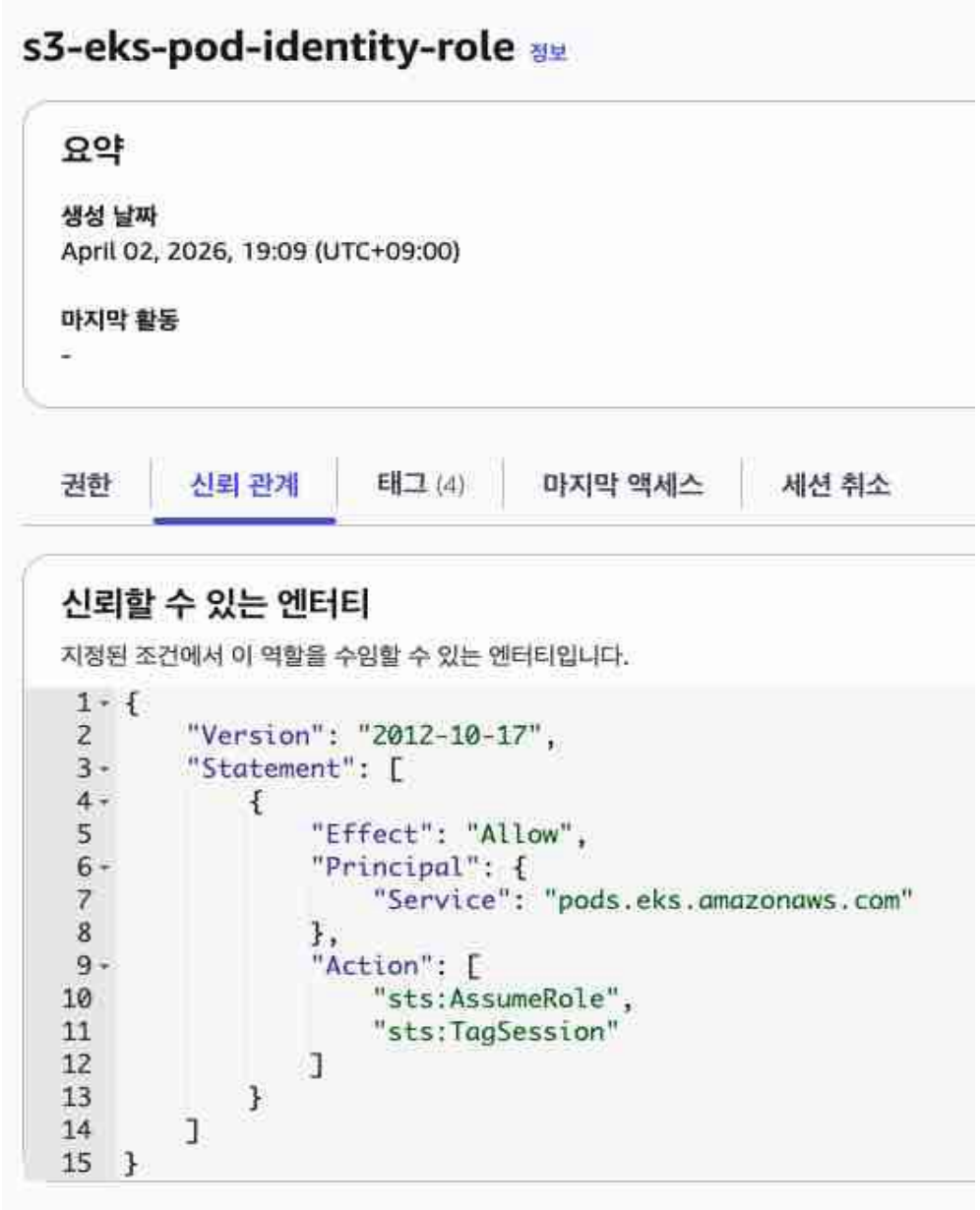
Pod Identity 운영 1.(IRSA IdP 방식과 비교)



Pod Identity 운영 2.(IRSA Trust Policy 구조 변경)



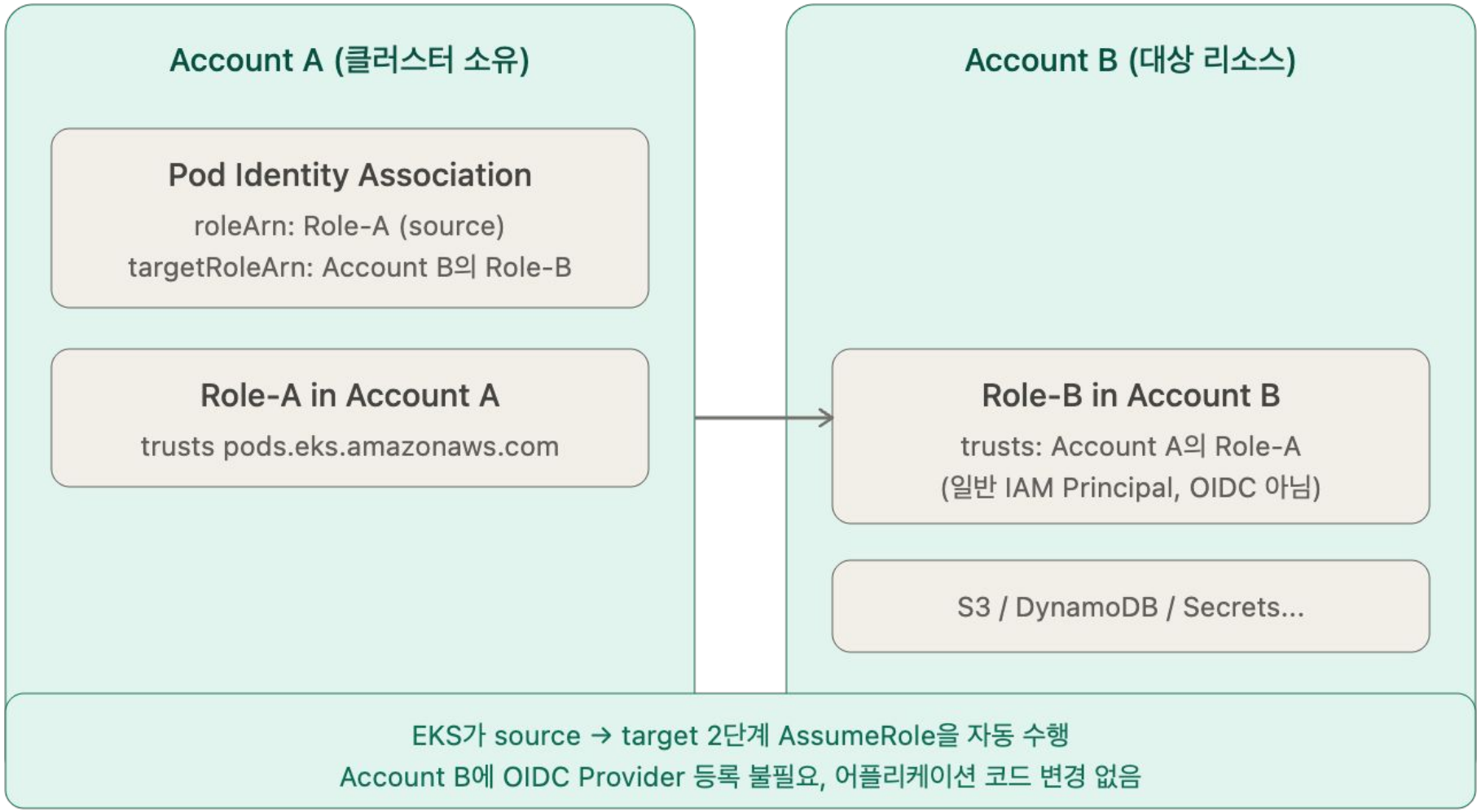
Pod Identity Trust Policy



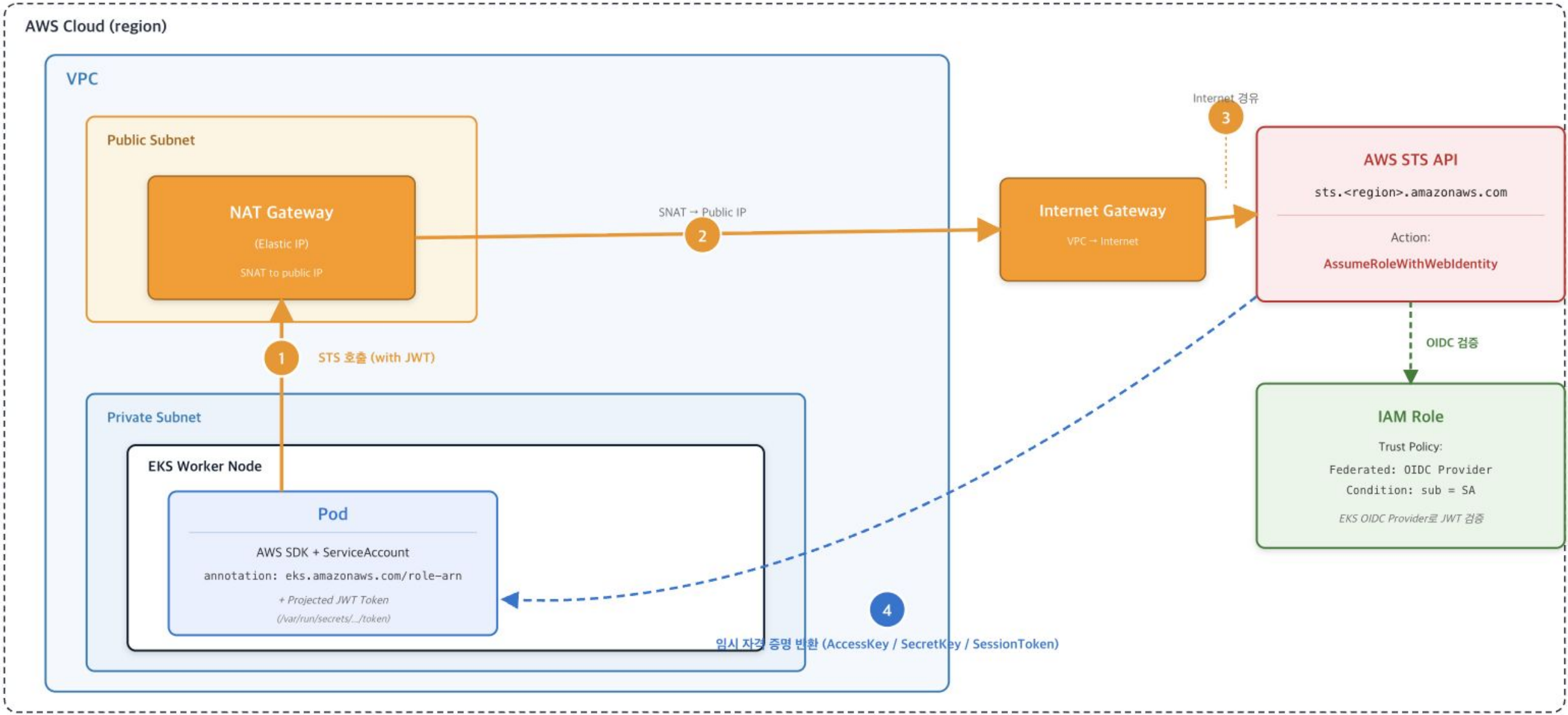
Pod Identity Trust Policy 예시

구조적으로 Sub / Aud 를 구분할 필요가 없어 해당 문제 해결

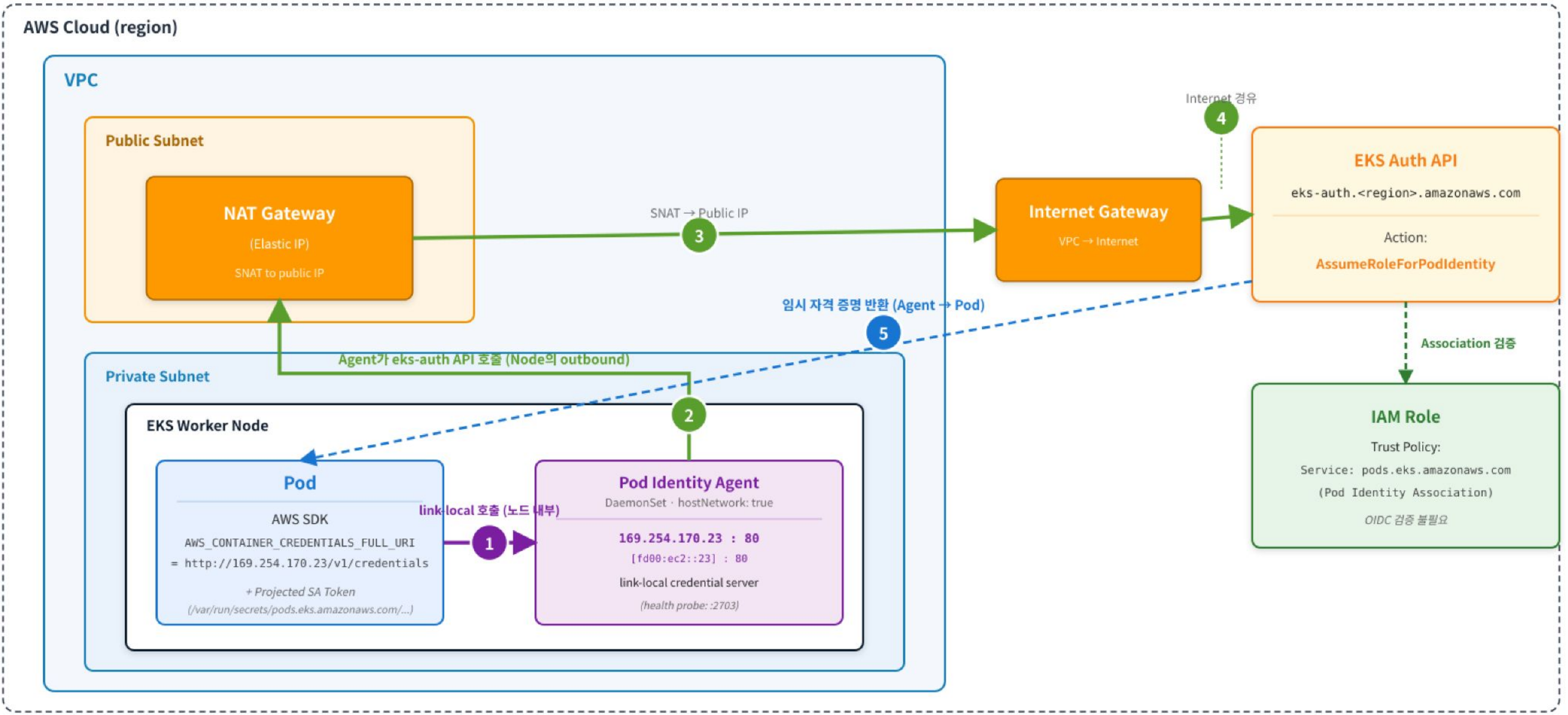
Pod Identity 운영 3.(IRSA Cross-Account 방식과 비교)



네트워크 흐름 – IRSA


AWS Cloud (region)
VPC
Public Subnet
NAT Gateway
(Elastic IP)
SNAT to public IP
SNAT → Public IP
2
Internet Gateway
VPC → Internet
Internet 경유
3
AWS STS API
sts.<region>.amazonaws.com
Action:
AssumeRoleWithWebIdentity
OIDC 검증
IAM Role
Trust Policy:
Federated: OIDC Provider
Condition: sub = SA
EKS OIDC Provider로 JWT 검증
1
STS 호출 (with JWT)
Private Subnet
EKS Worker Node
Pod
AWS SDK + ServiceAccount
annotation: eks.amazonaws.com/role-arn
+ Projected JWT Token
(/var/run/secrets/.../token)
4
임시 자격 증명 반환 (AccessKey / SecretKey / SessionToken)

네트워크 흐름 – Pod Identity



마이그레이션 고려사항

사전 요구사항

Pod Identity 사용 전 확인

- EKS 클러스터 버전 1.24 이상
- Pod Identity Agent Add-on 설치 (kube-system DaemonSet)
- 노드 IAM Role에 eks-auth:AssumeRoleForPodIdentity 권한 (AmazonEKSWorkerNodePolicy 포함)
- Pod Identity 지원 SDK 버전 사용
- Private subnet 사용 시 EKS Auth API용 PrivateLink 엔드포인트 구성

전환 전략 / 주의사항

점진적 전환 권장

- 기존 IAM Role의 trust policy에 pods.eks.amazonaws.com 서비스 주체를 추가해 OIDC와 공존 가능
- Pod Identity Webhook은 IRSA보다 Pod Identity를 우선 적용 (동시 설정 시 Pod Identity가 사용됨)
- Fargate 워크로드는 Pod Identity 미지원 → IRSA 유지 필요
- EKS Anywhere · self-managed K8s · ROSA 환경은 IRSA 유지
- eksctl utils migrate-to-pod-identity 명령으로 일괄 전환 가능

선택 기준 정리

IRSA를 선택하는 경우

- Fargate에서 동작하는 Pod이 포함된 클러스터
- EKS 1.24 미만 또는 SDK 업그레이드가 어려운 워크로드
- AWS MSK(관리형 카프카 서비스)
- 이미 IRSA로 운영 중이며 단일 클러스터 규모에서 trust policy/OIDC provider 한도 이슈가 없는 경우

Pod Identity를 선택하는 경우

- 다중 클러스터에서 동일 IAM Role을 재사용하고 싶을 때
- OIDC provider/trust policy 한도에 도달했거나 도달 예상 환경
- IAM 관리자와 클러스터 운영자가 분리되어 있어 trust policy 수정 협업 비용이 큰 경우
- **신규 EKS 클러스터를 셋업하는 일반적인 경우 (AWS의 권장 방향)**

IRSA vs Pod Identity 비교

항목	IRSA	Pod Identity
출시	2019	2023년 11월 (re:Invent 2023)
인증 API	STS AssumeRoleWithWebIdentity	EKS Auth AssumeRoleForPodIdentity
Trust Policy Principal	Federated: OIDC Provider ARN	Service: pods.eks.amazonaws.com
Trust Policy Action	sts:AssumeRoleWithWebIdentity	sts:AssumeRole + sts:TagSession
Token audience	sts.amazonaws.com	pods.eks.amazonaws.com (24h)
설정 위치	SA annotation + IAM trust policy	EKS API의 PodIdentityAssociation
주입 메커니즘	Pod Identity Webhook (mutating)	Pod Identity Agent (DaemonSet)
OIDC Provider	클러스터마다 IAM에 등록 필요	불필요
Role 재사용성	다른 클러스터에서 사용 시 trust policy 추가	동일 Role을 여러 클러스터에서 그대로 사용
Session Tags(ABAC)	기본 미지원	cluster/namespace/SA/pod 자동 주입
Fargate	지원	미지원 (DaemonSet 필요)
EKS 외 환경	EKS Anywhere · self-managed K8s · ROSA 가능	EKS in the cloud 전용
Cross Account 접근	상대 계정에 IdP 추가 및 절차 필요	association 생성 시 --target-role-arn 설정 (대상 계정 / 대상 Role 지정)

정리

01

정적 키 → 임시 자격증명

Access Key 직접 주입 → STS 기반 임시 자격증명으로 보안 모델 자체가 이동

02

노드 단위 → Pod 단위

Node Role 공유 → IRSA가 Kubernetes ServiceAccount 단위로 IAM 권한 분리

03

OIDC 중심 → EKS API 중심

Pod Identity는 신뢰 관계를 IAM trust policy(OIDC)에서 EKS Auth API로 옮겨 운영 단순화

Ref & QnA

- https://docs.aws.amazon.com/eks/latest/userguide/cross-account-access.html
- https://lonelynova.tistory.com/375
- https://hongchangsub.com/aws-eks-auth
- hcs4125@gmail.com